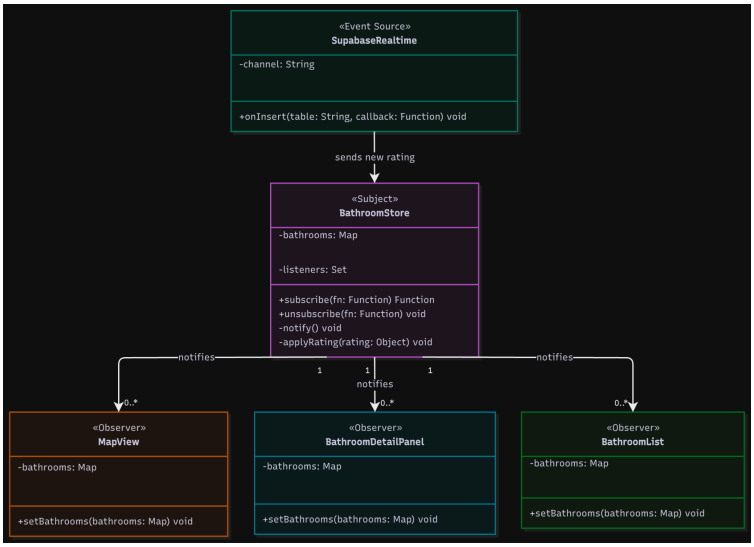
Design Patterns

- Name the pattern.
    - Observer
- Explain the problem it solves in your application.
    - Toilettracker shows cleanliness and busyness ratings in several places at once; when a new rating comes in, several parts of the UI need to update for everyone currently using the app . Without a pattern, each one would have to re fetch the data on its own, or the rating form would need to know about every other view and update each one by hand.
- Identify the classes, modules, components, or services involved.
    - Subject: BathroomStore, a frontend module that holds the current bathroom data and opens a single supabase realtime subscription to the ratings table.
    - Observers: MapView, BathroomDetailPanel, and BathroomList (React components) that subscribe to the store and re render when notified.
    - Trigger: Supabase Realtime, which sends a change event when the Express backend inserts a new rating
- Include a small UML diagram, code snippet, pseudocode, or annotated design sketch.
    - 

- Explain why this pattern is a better choice than placing all of the logic in one large component or module.
    - If one large component owned the map, the list, the detail panel, and the database subscription, every new view would mean editing that component, and a bug in one section could break the others. With Observer, the store only knows that it has listeners, not what they are, so each view stays independent. We can add a new view later (for example, a "favorites" panel) by subscribing it, without changing existing code. It also means each client keeps one realtime connection instead of one per component, which keeps us within Supabase's connection limits.
- Planned Implementation
    - Once the ratings table and the post/api/ratings endpoint are in place, we can turn on Realtime for the ratings table in Supabase and build BathroomStore as a

standalone module in the frontend. Then we can convert MapView and BathroomList to subscribe to the store instead of fetching data directly, and add BathroomDetailPanel as a third observer.